

FINCORE



Lending System

Manual ledgers create institutional risk. This architecture implements a 3NF relational model to automate the lending lifecycle. By integrating rigid SQL validations with real-time audit transparency, the system transforms fragmented data into a secure, scalable financial ecosystem.

Initial Project Concept & Database Design

Lending System

Presented by

Meerub Nadeem L24-5741

Hadia InayatUllah L24-5559

Maheera Yasir I24-5500

Syed Zain Ali L24-5618

Affaf Kashif L24-5508

BS-Fintech - 4B

Date

22 April 2026

Course

Intro to DBMS

Ref

Final Project

Abstract

VISION

A centralized relational database architecture designed to automate the end-to-end lifecycle of institutional lending, replacing fragmented manual record-keeping with a high-integrity digital repository.

DATA GOVERNANCE

The system manages complex multi-dimensional data, including borrower identities, multi-tiered repayment schedules, and physical collateral inventory, ensuring a "Single Source of Truth."

Abstract

Strategic Impact

Institutional Automation



No More Paperwork

Dynamic Risk Mitigation



Smart Safety Checks

Data Integrity/Fraud Prevention



Locked & Secure Data

Operational Transparency



Clear Progress Tracking

Elimination of Redundancy



No Repeated Records

Domain

Banking & Finance Sector

The system is built for the lending industry to manage large amounts of customer and loan data

Targeted User Base

Designed for bank staff, including loan officers and credit analysts, to handle daily operations

GUI Interaction

Staff use a visual interface to manage the database instead of writing manual SQL queries

Daily Operations

Supports onboarding new borrowers, logging monthly installments, and updating collateral valuations.

Smart Backend Tools

Includes high-level dashboards for tracking total loans issued, default rates, and security audits.

Active Tracking

Provides assessment views to help monitor struggling loans and identify high-risk borrowers.

Dataset

A	B	C	D	E	F	G	H	I	J
NIC	FirstName	LastName	PhoneNumber	Email	Address	DateRegistered	EmploymentStatus	MonthlyIncome	CreditScore
6338-2878459-7	Fatima	Gillani	0334-3232941	fatima.gillani2151@gm	House 169, Pha	2022-09-03	Business Owner	200004	61
6338-2878459-7	Fatima	Gillani	0334-3232941	fatima.gillani2151@gm	House 169, Pha	2022-09-03	Business Owner	200004	61
6338-2878459-7	Fatima	Gillani	0334-3232941	fatima.gillani2151@gm	House 169, Pha	2022-09-03	Business Owner	200004	61

K	L	M	N	O	P	Q	R
LoanTypeName	RequestedAmount	ApplicationDate	ApprovedAmount	InterestRate	LoanDurationMonth	CollateralAssetType	CollateralEstimatedValue
51 Education	505275	2022-09-18	505275	8	12	Bonds	509289.95
51 Education	505275	2022-09-18	505275	8	12	Bonds	509289.95
51 Education	505275	2022-09-18	505275	8	12	Bonds	509289.95

S	T	U	V	W	X	Y	Z	AA	AB	AC
LoanStartDate	LoanEndDate	InstallmentNumber	DueDate	ScheduledInstallmentAm	PaymentDate	PaymentAmount	PaymentMethod	IsRescheduled	LatePaymentFine	ReasonForDe
2022-09-25	2023-09-20	1	2022-10-25	45474.75	2022-10-24	45474.75	Cheque	Yes		0 Medical
2022-09-25	2023-09-20	2	2022-11-24	45474.75	2022-11-28	45474.75	Cash	Yes	5000	Medical
2022-09-25	2023-09-20	3	2022-12-24	45474.75	2022-12-24	45474.75	Cash	Yes		0 Medical
2022-09-25	2023-09-20	4	2023-01-23	45474.75	2023-01-30	45474.75	Cheque	Yes	5000	Medical

Functional

01.

Borrower Management

Complete lifecycle control through CRUD operations, enabling staff to onboard users, update profiles, and maintain accurate credit histories.

02.

Loan & Asset Tracking

Formal recording of approved contracts and physical collateral details, linking assets like real estate or vehicles to specific loan accounts.

03.

Transaction Engine

Automated logging of monthly installments with real-time tracking of payment dates, methods, and overdue penalty calculations.

Functional

04.

Risk Classification

Algorithmic logic that automatically categorizes borrowers into Low, Medium, or High-risk groups based on credit scores and income stability.

05.

Integrity Auditing

Security-driven triggers that capture every update or deletion in sensitive tables, creating an immutable history to prevent financial fraud.

06.

Analytics Dashboard

Backend business intelligence reporting that generates summaries of interest revenue and identifies high-priority overdue accounts.

Non-Functional

01.

System Security

Strict role-based access control ensuring only authorized personnel can interact with data, backed by automated accountability logs.

02.

Data Integrity

Enforcement of Relational Integrity through primary/foreign keys and ACID compliance to prevent corrupt or disconnected data entries.

03.

Performance Speed

Optimized query execution for real-time joins and views, ensuring low-latency results even when processing thousands of transaction rows.

Non-Functional

04.

Scalability

Future-proof database architecture designed to handle rapid data expansion without requiring a structural overhaul of the core tables.

05.

High Availability

Mission-critical operational uptime specifically optimized for banking hours to ensure seamless transaction processing for staff.

06.

Maintainability

A clean, normalized schema that allows administrators to integrate new loan types or asset categories with minimal configuration changes.

Design Assumptions

The foundational business rules and operational boundaries established to standardize data logic and ensure architectural reliability.



CNIC Uniqueness

Each borrower is identified by a unique, government-issued 13-digit CNIC which serves as the natural key



Collateral Liquidity

Assumes all pledged assets have verifiable market values that can be appraised during the application phase.



Currency Logic

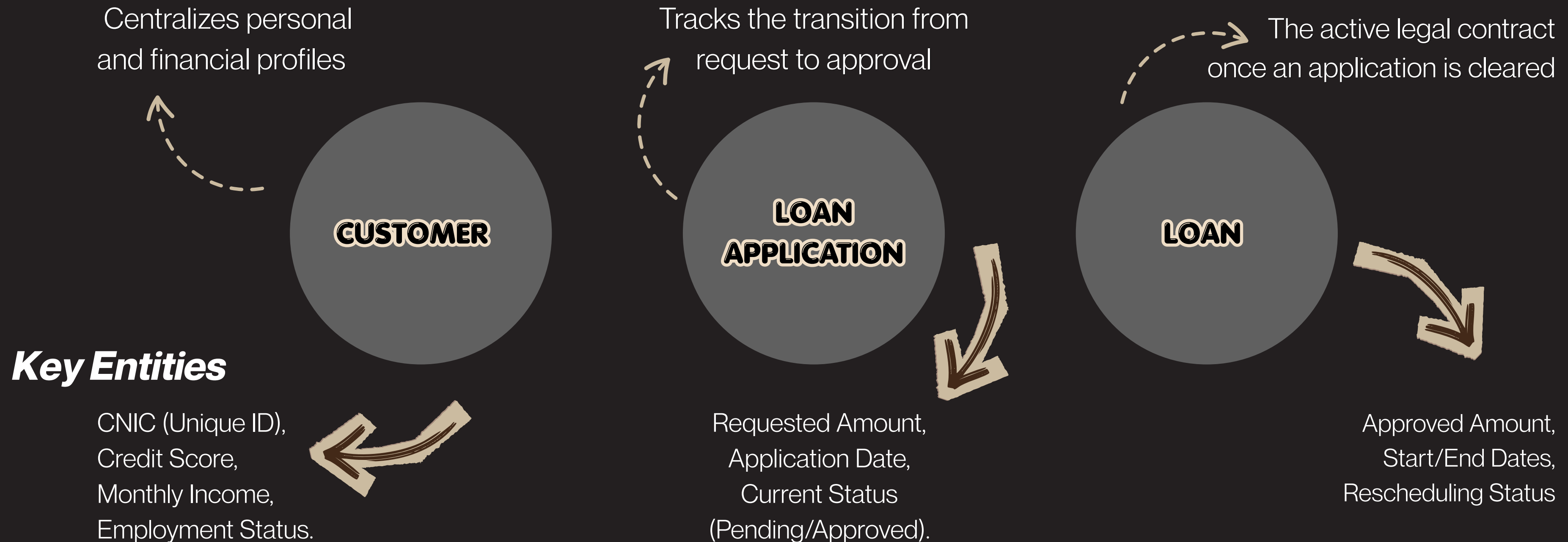
All financial records and interest calculations are performed in a single standardized currency (PKR).



Staff Competency

System assumes users are trained to operate the GUI and input verified data from official documents.

Primary Entities



Support & Security Entities

The main transaction ledger for installment tracking.

Manages physical assets pledged as security

An automated "security camera" for the database.

PAYMENT

COLLATERAL

AUDIT LOG

Key Entities

Installment Number,
Due Date,
Actual Payment Date, Late Fines

Asset Type (Real Estate, Vehicle, etc.),
Estimated Market Value

Table Name, Action (Update/Delete),
Old vs. New Values, Timestamp.

Cardinality

- **Customer → Loan Application (1:N)**

- One customer can apply for multiple loans over time, but each application belongs to only one specific customer.

- **Loan Application → Loan (1:1)**

- Once an application is approved, it converts into exactly one active loan contract.

- **Loan → Payment (1:N)**

- A single loan is paid back through multiple monthly installments recorded in the payment ledger.

- **Loan → Collateral (1:1)**

- To maintain strict security, each loan is backed by one specific high-value asset (like a car or property).

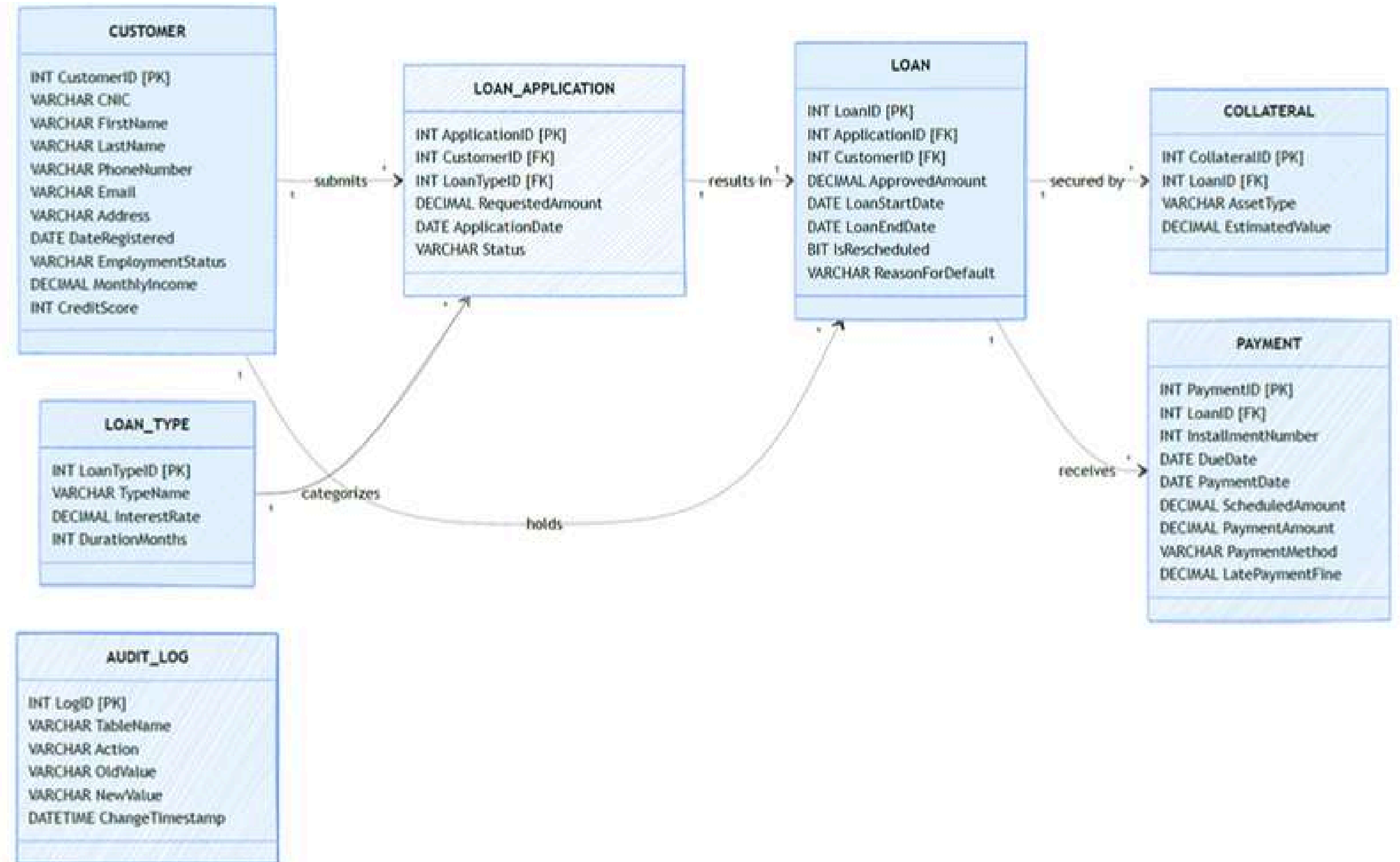
- **Loan → Loan Type (N:1)**

- Many different loans can fall under a single category (e.g., several customers having "Home Loans").

- **System → Audit Log (1:N)**

- The system generates many security logs for every sensitive change made to the database tables.

ERD



```
CREATE TABLE CUSTOMER (  
    CustomerID INT IDENTITY(1,1) PRIMARY KEY,  
    CNIC VARCHAR(15) NOT NULL UNIQUE,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    PhoneNumber VARCHAR(15) NOT NULL UNIQUE,  
    Email VARCHAR(100) NOT NULL UNIQUE,  
    Address VARCHAR(255),  
    DateRegistered DATE NOT NULL,  
    EmploymentStatus VARCHAR(50),  
    MonthlyIncome DECIMAL(18,2),  
    CreditScore INT CHECK (CreditScore BETWEEN 300 AND 850)  
);
```

```
CREATE TABLE AUDIT_LOG (  
    LogID INT IDENTITY(1,1) PRIMARY KEY,  
    TableName VARCHAR(50) NOT NULL,  
    ActionType VARCHAR(100) NOT NULL,  
    RecordID INT NOT NULL,  
    ActionDate DATETIME DEFAULT GETDATE(),  
    OldValue VARCHAR(MAX),  
    NewValue VARCHAR(MAX));
```

```
CREATE TABLE LOAN (  
    LoanID INT IDENTITY(1,1) PRIMARY KEY,  
    ApplicationID INT NOT NULL FOREIGN KEY REFERENCES LOAN_APPLICATION(ApplicationID),  
    CustomerID INT NOT NULL FOREIGN KEY REFERENCES CUSTOMER(CustomerID),  
    ApprovedAmount DECIMAL(18,2) NOT NULL CHECK (ApprovedAmount > 0),  
    LoanStartDate DATE NOT NULL,  
    LoanEndDate DATE NOT NULL,  
    IsRescheduled BIT DEFAULT 0, -- 1 for Yes, 0 for No  
    ReasonForDefault VARCHAR(100),  
    CONSTRAINT CHK_Dates CHECK (LoanEndDate > LoanStartDate)  
);
```

```
CREATE TABLE PAYMENT (  
    PaymentID INT IDENTITY(1,1) PRIMARY KEY,  
    LoanID INT NOT NULL FOREIGN KEY REFERENCES LOAN(LoanID),  
    InstallmentNumber INT NOT NULL,  
    DueDate DATE NOT NULL,  
    PaymentDate DATE NULL,  
    ScheduledAmount DECIMAL(18,2) NOT NULL,  
    PaymentAmount DECIMAL(18,2) NOT NULL CHECK (PaymentAmount >= 0),  
    LatePaymentFine DECIMAL(18,2) DEFAULT 0);
```

```
CREATE TABLE LOAN_APPLICATION (  
    ApplicationID INT IDENTITY(1,1) PRIMARY KEY,  
    CustomerID INT NOT NULL FOREIGN KEY REFERENCES CUSTOMER(CustomerID),  
    LoanTypeID INT NOT NULL FOREIGN KEY REFERENCES LOAN_TYPE(LoanTypeID),  
    RequestedAmount DECIMAL(18,2) NOT NULL CHECK (RequestedAmount > 0),  
    ApplicationDate DATE NOT NULL,  
    Status VARCHAR(20) DEFAULT 'Approved'  
);
```

```
CREATE TABLE LOAN_TYPE (  
    LoanTypeID INT IDENTITY(1,1) PRIMARY KEY,  
    TypeName VARCHAR(50) NOT NULL,  
    InterestRate DECIMAL(5,2) NOT NULL CHECK (InterestRate > 0)  
    DurationMonths INT NOT NULL
```

```
CREATE TABLE COLLATERAL (  
    CollateralID INT IDENTITY(1,1) PRIMARY KEY,  
    LoanID INT NOT NULL FOREIGN KEY REFERENCES LOAN(LoanID),  
    AssetType VARCHAR(50) NOT NULL,  
    EstimatedValue DECIMAL(18,2) NOT NULL);
```

```
INSERT INTO CUSTOMER (CNIC, FirstName, LastName, PhoneNumber, Email, Address, DateRegistered, EmploymentStatus, MonthlyIncome, CreditScore)
SELECT DISTINCT CNIC, FirstName, LastName, PhoneNumber, Email, Address, DateRegistered, EmploymentStatus, MonthlyIncome, CreditScore
FROM Raw_Loan_Data_1052
WHERE CNIC IS NOT NULL;
```

```
INSERT INTO LOAN_TYPE (TypeName, InterestRate, DurationMonths)
SELECT DISTINCT LoanTypeName, InterestRate, LoanDurationMonths
FROM Raw_Loan_Data_1052
WHERE LoanTypeName IS NOT NULL;
```

```
INSERT INTO LOAN_APPLICATION (CustomerID, LoanTypeID, RequestedAmount, ApplicationDate, Status)
SELECT DISTINCT c.CustomerID, lt.LoanTypeID, r.RequestedAmount, r.ApplicationDate, 'Approved'
FROM Raw_Loan_Data_1052 r
JOIN CUSTOMER c ON r.CNIC = c.CNIC
JOIN LOAN_TYPE lt ON r.LoanTypeName = lt.TypeName AND r.InterestRate = lt.InterestRate AND r.LoanDurationMonths = lt.DurationMonths
WHERE r.RequestedAmount IS NOT NULL;
```

```
INSERT INTO COLLATERAL (LoanID, AssetType, EstimatedValue)
SELECT DISTINCT l.LoanID, r.CollateralAssetType, r.CollateralEstimatedValue
FROM Raw_Loan_Data_1052 r
JOIN CUSTOMER c ON r.CNIC = c.CNIC
JOIN LOAN l ON l.CustomerID = c.CustomerID AND l.ApprovedAmount = r.ApprovedAmount AND l.LoanStartDate = r.LoanStartDate
WHERE r.CollateralAssetType IS NOT NULL AND r.CollateralAssetType != 'None';
```

```
INSERT INTO LOAN (ApplicationID, CustomerID, ApprovedAmount, LoanStartDate, LoanEndDate, IsRescheduled, ReasonForDefault)
SELECT DISTINCT
    a.ApplicationID,
    c.CustomerID,
    r.ApprovedAmount,
    r.LoanStartDate,
    r.LoanEndDate,

    CASE WHEN CAST(r.IsRescheduled AS VARCHAR(10)) IN ('Yes', '1', 'True') THEN 1 ELSE 0 END,
    r.ReasonForDefault
FROM Raw_Loan_Data_1052 r
JOIN CUSTOMER c ON r.CNIC = c.CNIC
JOIN LOAN_APPLICATION a ON a.CustomerID = c.CustomerID AND a.RequestedAmount = r.RequestedAmount AND a.ApplicationDate = r.ApplicationDate
WHERE r.ApprovedAmount IS NOT NULL;
```

```
INSERT INTO PAYMENT (LoanID, InstallmentNumber, DueDate, PaymentDate, ScheduledAmount, PaymentAmount, PaymentMethod, LatePaymentFine)
SELECT DISTINCT
    l.LoanID,
    r.InstallmentNumber,
    r.DueDate,
    r.PaymentDate,
    r.ScheduledInstallmentAmount,
    r.PaymentAmount,
    r.PaymentMethod,
    r.LatePaymentFine
FROM Raw_Loan_Data_1052 r
JOIN CUSTOMER c ON r.CNIC = c.CNIC
JOIN LOAN l ON l.CustomerID = c.CustomerID
WHERE r.InstallmentNumber IS NOT NULL;
```

```
CREATE OR ALTER TRIGGER trg_Audit_Loan_Updates
ON LOAN
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF UPDATE(ApprovedAmount)
    BEGIN
        INSERT INTO AUDIT_LOG (TableName, ActionType, RecordID, ActionDate, OldValue, NewValue)
        SELECT
            'LOAN',
            'UPDATE - Approved Amount',
            CAST(i.LoanID AS VARCHAR(50)),
            GETDATE(),
            CAST(d.ApprovedAmount AS VARCHAR(50)),
            CAST(i.ApprovedAmount AS VARCHAR(50))
        FROM inserted i
        JOIN deleted d ON i.LoanID = d.LoanID
        WHERE i.ApprovedAmount <> d.ApprovedAmount;
    END
    |
    IF UPDATE(IsRescheduled)
    BEGIN
        INSERT INTO AUDIT_LOG (TableName, ActionType, RecordID, ActionDate, OldValue, NewValue)
        SELECT
            'LOAN',
            'UPDATE - Reschedule Status',
            CAST(i.LoanID AS VARCHAR(50)),
            GETDATE(),
            CAST(d.IsRescheduled AS VARCHAR(10)),
            CAST(i.IsRescheduled AS VARCHAR(10))
        FROM inserted i
        JOIN deleted d ON i.LoanID = d.LoanID
        WHERE i.IsRescheduled <> d.IsRescheduled;
    END
END;
```

```
CREATE OR ALTER TRIGGER trg_Auto_Credit_Penalty
ON PAYMENT
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF UPDATE(LatePaymentFine)
    BEGIN
        UPDATE C
        SET C.CreditScore = CASE
            WHEN C.CreditScore - 15 < 300 THEN 300
            ELSE C.CreditScore - 15
        END
        FROM CUSTOMER C
        JOIN LOAN L ON C.CustomerID = L.CustomerID
        JOIN inserted i ON L.LoanID = i.LoanID
        JOIN deleted d ON i.PaymentID = d.PaymentID
        WHERE i.LatePaymentFine > d.LatePaymentFine;
    END
END;
GO
```

```
CREATE OR ALTER TRIGGER trg_Audit_Loan_Insertion
ON LOAN
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO AUDIT_LOG (TableName, ActionType, RecordID, ActionDate, OldValue, NewValue)
    SELECT
        'LOAN',
        'INSERT - New Loan Issued',
        CAST(i.LoanID AS VARCHAR(50)),
        GETDATE(),
        'NULL',
        'Amount: ' + CAST(i.ApprovedAmount AS VARCHAR(20)) + ' | CustID: ' + CAST(i.CustomerID AS VARCHAR(10))
    FROM inserted i;
END;
GO
```

```
CREATE VIEW vw_Customer_Late_Fines
AS
SELECT
    C.CNIC,
    C.FirstName + ' ' + C.LastName AS CustomerName,
    C.PhoneNumber,
    L.ApprovedAmount AS TotalLoanAmount,
    COUNT(P.PaymentID) AS TotalPaymentsMade,
    SUM(P.LatePaymentFine) AS TotalFinesAccumulated
FROM CUSTOMER C
JOIN LOAN L ON C.CustomerID = L.CustomerID
JOIN PAYMENT P ON L.LoanID = P.LoanID
GROUP BY
    C.CNIC,
    C.FirstName,
    C.LastName,
    C.PhoneNumber,
    L.ApprovedAmount
HAVING SUM(P.LatePaymentFine) > 0;
GO
```

```
SELECT * FROM vw_Customer_Late_Fines
ORDER BY TotalFinesAccumulated DESC;
```

```
CREATE VIEW vw_Risk_Classification
AS
SELECT
    CustomerID,
    CNIC,
    FirstName + ' ' + LastName AS FullName,
    CreditScore,
    MonthlyIncome,
    CASE
        WHEN CreditScore >= 750 AND MonthlyIncome > 100000 THEN 'Low Risk'
        WHEN CreditScore >= 600 THEN 'Medium Risk'
        ELSE 'High Risk'
    END AS RiskLevel
FROM CUSTOMER;
GO

SELECT * FROM vw_Risk_Classification
WHERE RiskLevel = 'High Risk';
```

```
CREATE VIEW vw_Bank_Dashboard_Stats AS
SELECT
    (SELECT COUNT(*) FROM CUSTOMER) AS TotalCustomers,
    (SELECT SUM(ApprovedAmount) FROM LOAN) AS TotalMoneyLent,
    (SELECT SUM(LatePaymentFine) FROM PAYMENT) AS TotalFinesAccumulated,
    (SELECT COUNT(*) FROM LOAN WHERE IsRescheduled = 1) AS RescheduledLoans,
    (SELECT COUNT(*) FROM COLLATERAL) AS SecuredLoans
GO
```

```
SELECT * FROM vw_Bank_Dashboard_Stats;
```

```
CREATE VIEW vw_Defaulter_Watchlist AS
SELECT
    C.FirstName + ' ' + C.LastName AS DefaulterName,
    C.PhoneNumber,
    C.CreditScore,
    L.ApprovedAmount AS MoneyOwed,
    SUM(P.LatePaymentFine) AS TotalFines,
    COL.AssetType AS AssetToSeize,
    COL.EstimatedValue AS AssetValue
FROM CUSTOMER C
JOIN LOAN L ON C.CustomerID = L.CustomerID
JOIN PAYMENT P ON L.LoanID = P.LoanID
JOIN COLLATERAL COL ON L.LoanID = COL.LoanID
WHERE C.CreditScore < 600
GROUP BY
    C.FirstName, C.LastName, C.PhoneNumber, C.CreditScore,
    L.ApprovedAmount, COL.AssetType, COL.EstimatedValue
HAVING SUM(P.LatePaymentFine) > 10000; -- Only show people with massive fines
GO

select * from vw_Defaulter_Watchlist;

SELECT TOP 1 * FROM vw_Defaulter_Watchlist;
```

```
CREATE PROCEDURE sp_Add_Customer
    @CNIC VARCHAR(15),
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @PhoneNumber VARCHAR(15),
    @Email VARCHAR(100),
    @Address VARCHAR(255),
    @EmploymentStatus VARCHAR(50),
    @MonthlyIncome DECIMAL(18,2),
    @CreditScore INT
```

```
AS
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    INSERT INTO CUSTOMER (CNIC, FirstName, LastName, PhoneNumber, Email, Address, DateRegistered, EmploymentStatus, MonthlyIncome)
    VALUES (@CNIC, @FirstName, @LastName, @PhoneNumber, @Email, @Address, GETDATE(), @EmploymentStatus, @MonthlyIncome)
```

```
    SELECT SCOPE_IDENTITY() AS NewCustomerID;
```

```
CREATE PROCEDURE sp_ForgiveLateFines
    @TargetLoanID INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @TotalFines DECIMAL(18,2);

    SELECT @TotalFines = SUM(LatePaymentFine)
    FROM PAYMENT
    WHERE LoanID = @TargetLoanID;

    IF @TotalFines = 0 OR @TotalFines IS NULL
    BEGIN
        PRINT 'No late fines found for Loan ID ' + CAST(@TargetLoanID AS VARCHAR(10)) + '.';
        RETURN;
    END

    UPDATE PAYMENT
    SET LatePaymentFine = 0
    WHERE LoanID = @TargetLoanID;

    PRINT 'SUCCESS! A total of Rs. ' + CAST(@TotalFines AS VARCHAR(20)) + ' in fines has been forgiven.';
END;
GO

EXEC sp_ForgiveLateFines @TargetLoanID = 15;
```



```
CREATE OR ALTER PROCEDURE sp_Apply_For_Loan
    @CustomerID INT,
    @RequestedAmount DECIMAL(18,2),
    @DurationMonths INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @CreditScore INT, @MonthlyIncome DECIMAL(18,2), @MonthlyInstallment DECIMAL(18,2);
    DECLARE @NewAppID INT, @NewLoanID INT, @LoanType INT;

    SELECT @CreditScore = CreditScore, @MonthlyIncome = MonthlyIncome FROM CUSTOMER WHERE CustomerID = @CustomerID;

    IF @CreditScore IS NULL
    BEGIN
        SELECT 'REJECTED' AS Status, 'Error: Customer ID not found.' AS Message; RETURN;
    END

    SELECT TOP 1 @LoanType = LoanTypeID FROM LOAN_TYPE WHERE DurationMonths = @DurationMonths;
    IF @LoanType IS NULL SET @LoanType = 1;

    SET @MonthlyInstallment = @RequestedAmount / @DurationMonths;

    IF @CreditScore < 500
    BEGIN
        SELECT 'REJECTED' AS Status, 'Credit Score is below minimum 500.' AS Message; RETURN;
    END

    IF @MonthlyInstallment > (@MonthlyIncome * 0.40)
    BEGIN
        SELECT 'REJECTED' AS Status, 'Installment exceeds 40% of income.' AS Message; RETURN;
    END

    INSERT INTO LOAN_APPLICATION (CustomerID, LoanTypeID, RequestedAmount, ApplicationDate, Status)
    VALUES (@CustomerID, @LoanType, @RequestedAmount, CAST(GETDATE() AS DATE), 'Approved');
    SET @NewAppID = SCOPE_IDENTITY();

    INSERT INTO LOAN (ApplicationID, CustomerID, ApprovedAmount, LoanStartDate, LoanEndDate, IsRescheduled)
    VALUES (@NewAppID, @CustomerID, @RequestedAmount, CAST(GETDATE() AS DATE), CAST(DATEADD(MONTH, @DurationMonths, GETDATE()) AS DATE), 'False');

    SET @NewLoanID = SCOPE_IDENTITY();
    EXEC sp_Generate_EMI_Schedule @LoanID = @NewLoanID, @Principal = @RequestedAmount, @DurationMonths = @DurationMonths;

    SELECT 'APPROVED' AS Status, 'Loan issued & EMI Schedule generated!' AS Message;
END;
GO
```

```
-CREATE OR ALTER PROCEDURE sp_Process_Payment
    @LoanID INT,
    @AmountPaid DECIMAL(18,2),
    @Method VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @NextInstallment INT;
    DECLARE @CalculatedDueDate DATETIME;
    DECLARE @ScheduledAmt DECIMAL(18,2);

    SELECT @NextInstallment = ISNULL(MAX(InstallmentNumber), 0) + 1
    FROM PAYMENT
    WHERE LoanID = @LoanID;

    SET @CalculatedDueDate = GETDATE();
    SET @ScheduledAmt = @AmountPaid;

    INSERT INTO PAYMENT (LoanID, InstallmentNumber, DueDate, ScheduledAmount, PaymentAmount, PaymentDate, PaymentMethod)
    VALUES (@LoanID, @NextInstallment, @CalculatedDueDate, @ScheduledAmt, @AmountPaid, GETDATE(), @Method);

END;
GO
```

```
CREATE OR ALTER PROCEDURE sp_Generate_EMI_Schedule
    @LoanID INT,
    @Principal DECIMAL(18,2),
    @DurationMonths INT,
    @AnnualInterestRate DECIMAL(5,2) = 15.00
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @MonthlyRate FLOAT = (@AnnualInterestRate / 100.0) / 12.0;
    DECLARE @EMI DECIMAL(18,2);
    DECLARE @Counter INT = 1;
    DECLARE @DueDate DATE = GETDATE();

    SET @EMI = @Principal * ( @MonthlyRate * POWER(1 + @MonthlyRate, @DurationMonths) )
        / ( POWER(1 + @MonthlyRate, @DurationMonths) - 1 );

    WHILE @Counter <= @DurationMonths
    BEGIN
        SET @DueDate = DATEADD(MONTH, 1, @DueDate);

        INSERT INTO PAYMENT (LoanID, InstallmentNumber, DueDate, ScheduledAmount, PaymentAmount, PaymentMethod)
        VALUES (@LoanID, @Counter, @DueDate, @EMI, 0, 'Pending');

        SET @Counter = @Counter + 1;
    END
END;
GO
```

Validations and Business Logic

Business Logic

Credit Eligibility: Applicants with a credit score under 500 must be blocked from applying.

Identity Verification: Every borrower must provide a valid, exactly 13-digit identity number

Financial Safety: Monthly income and loan amounts cannot be zero or negative.

Database Implementation

CHECK Constraint: Enforced on the CreditScore column in the CUSTOMER table.

LEN() Function: Used within a Constraint to validate the CNIC string length.

Non-Negative Constraints: Applied to MonthlyIncome and LoanAmount fields.

Validations and Business Logic

Business Logic

Security Accountability: Any change to a borrower's financial status must be recorded.

Penalty Enforcement: Late payments must incur a financial penalty automatically.

Data Cleanup: Deleting a customer must also clear their pending applications.

Database Implementation

AFTER UPDATE Trigger:

Automatically captures the old and new data into the AuditLog.

Computed Column / Trigger:

Calculates the LateFine based on DueDate logic

ON DELETE CASCADE: Maintains referential integrity between linked tables.

System Operational Flow

Application

Logging requests and mapping them to the LOAN_TYPE catalog



Servicing

Processing payments via the PAYMENT engine with automated fine logic.



Establishing verified borrower profiles in the CUSTOMER table.

Onboarding



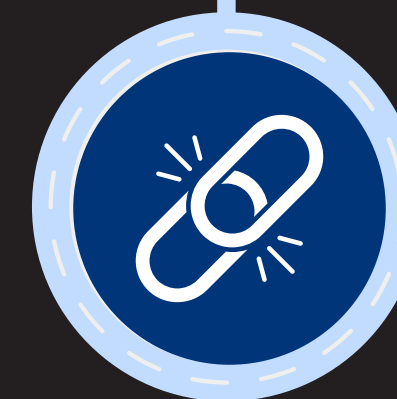
Validation

Approving contracts and securing physical assets in COLLATERAL

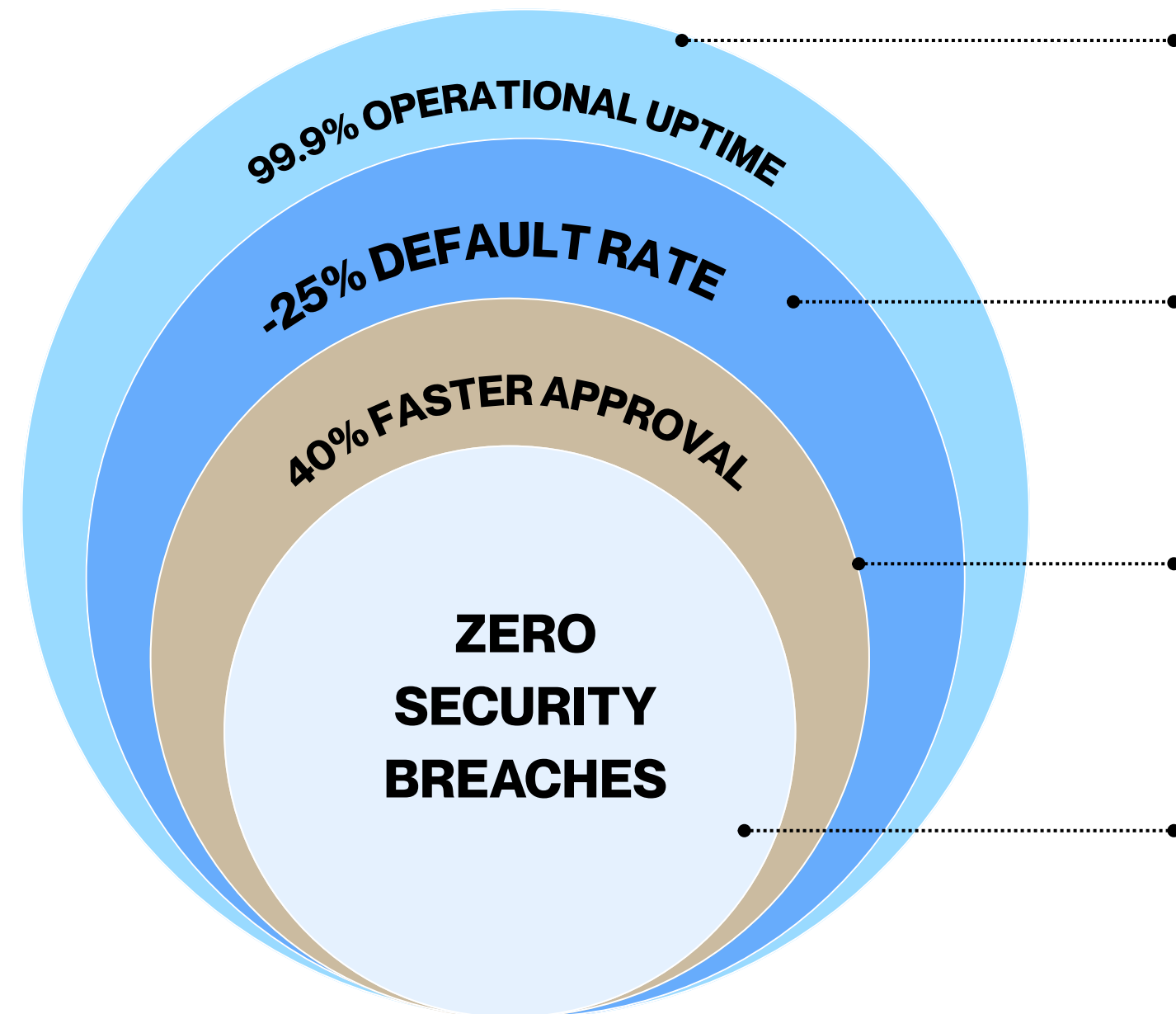


Auditing

Real-time monitoring via SQL Triggers and the AUDIT_LOG



Future Roadmap/Recommendations



Cloud Migration (Azure SQL)

Transitioning to a cloud environment to ensure 24/7 availability, automated daily backups, and global disaster recovery.

Proactive Alert Gateway:

Integration with an SMS/Email API to automatically notify borrowers 3 days before their DueDate, reducing default rates

Predictive AI Integration:

Utilizing historical PAYMENT data to train Machine Learning models that forecast default probability during the application stage.

Granular Security (RBAC):

Implementing Role-Based Access Control so junior staff can only log payments, while sensitive AUDIT_LOG access is restricted to managers.

Conclusion & System Impact

By automating the transition from manual ledgers to a secure relational architecture, we ensure institutional reliability and data-driven intelligence.



Relational Excellence:

Transitioning from manual risks to a 3rd Normal Form (3NF) architecture to ensure 100% data reliability.



Operational

Automation: Automated ledgers with a Self-Monitoring System that handles installments and fines automatically.



Institutional Security:

Guaranteeing accountability through an Insert-Only Audit Log, protecting the bank against data tampering.

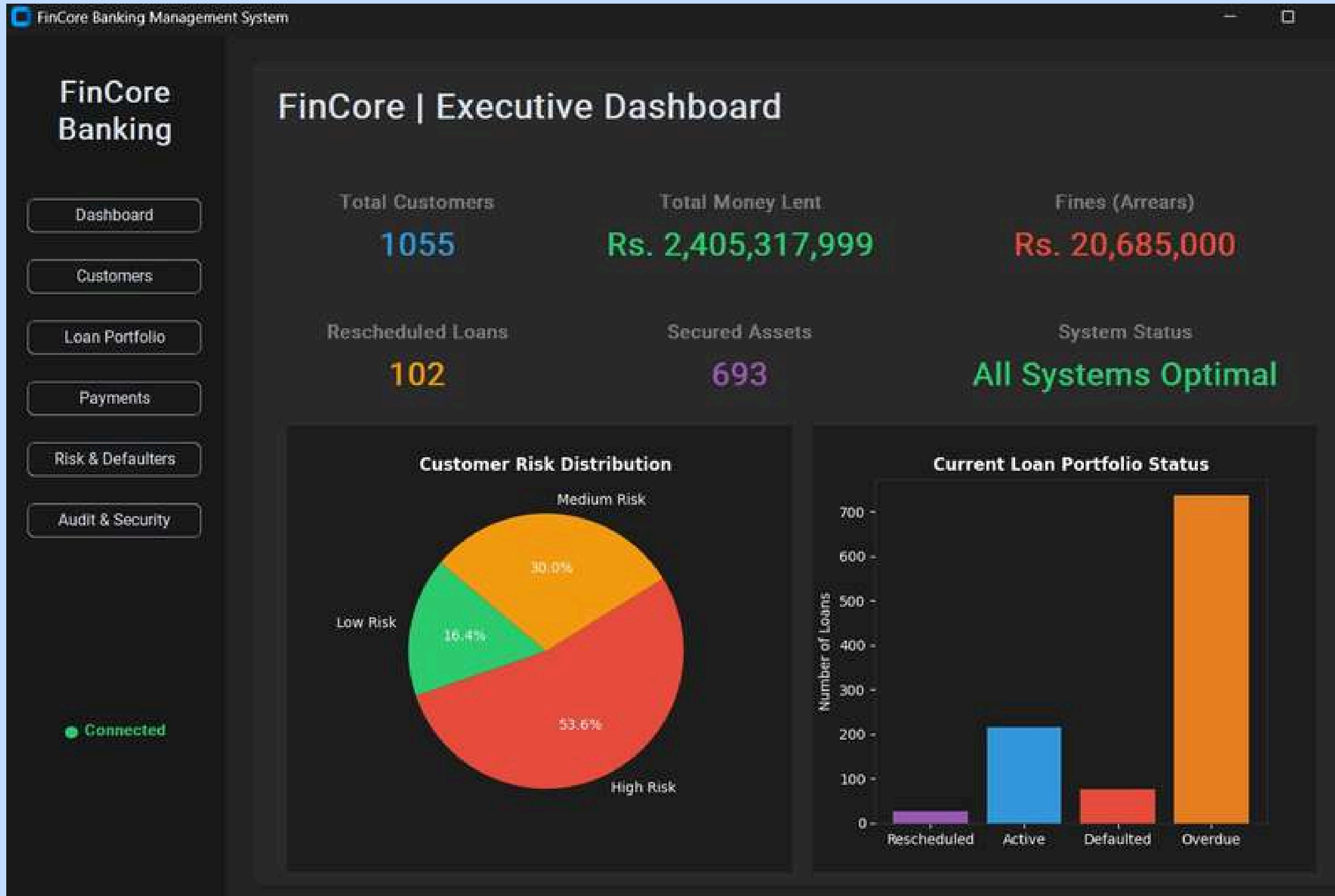
OUR SYSTEM DOESN'T
JUST STORE DATA: IT
SECURES THE
INSTITUTION'S
FUTURE THROUGH
AUTOMATED RISK
CONTROL AND DATA-
DRIVEN
INTELLIGENCE!



GRAPHICAL USER INTERFACE



MAIN DASHBOARD



CUSTOMER LIST

FinCore Banking Management System

FinCore Banking

Dashboard

Customers

Loan Portfolio

Payments

Risk & Defaulters

Audit & Security

Connected

Customer CRM

Issue New Loan

Add New Customer

Search by Name, CNIC, or Phone...

Search

ID	Name	CNIC	Phone	
1056	Hadia InayatUllah	78901-1234567-0	0323-4256929	View Profile
1055	Nasir Khan Jan	42101-0667432-3	0318-2773874	View Profile
1054	Meerub Nadeem	35201-0675153-2	0321-5438921	View Profile
1052	Fatima Ali	38995-9553320-2	0348-3732363	View Profile
1051	Bilal Tariq	38980-7091086-5	0346-6028720	View Profile
1050	Kiran Khan	38977-1215217-4	0316-4374498	View Profile
1049	Kiran Raza	38976-9718617-4	0330-5712171	View Profile
1048	Aisha Ali	38975-3460185-3	0346-7513191	View Profile
1047	Nida Chaudhry	38962-5906699-9	0300-3082761	View Profile
1046	Sadia Ali	38955-5041172-8	0346-2359797	View Profile
1045	Sana Akhtar	38947-8907735-7	0323-7458620	View Profile
1044	Fatima Shah	38944-6365624-3	0333-3094239	View Profile
1043	Nida Ali	38881-5453023-3	0312-8076883	View Profile
1042	Nadia Shah	38866-6907530-3	0313-2632913	View Profile



SEARCHING A CUSTOMER

FinCore Banking Management System

FinCore Banking

Dashboard

Customers

Loan Portfolio

Payments

Risk & Defaulters

Audit & Security

Connected

Customer CRM

Issue New Loan

Add New Customer

Zain Ali

Search

ID	Name	CNIC	Phone
214	Zain Ali	32689-2023772-4	0310-7509812

View Profile

ADDING A NEW CUSTOMER

FinCore Banking Management System

FinCore Banking

Dashboard

Customers

Loan Portfolio

Payments

Risk & Defaulters

Audit & Security

● Connected

Register New Customer

Customer Onboarding

First Name:

Last Name:

CNIC (e.g. 12345-1234567-1):

Phone Number:

Email Address:

Physical Address:

Employment Status (e.g. Salaried, Business):

Save Customer Record

Issue New Loan

+ Add

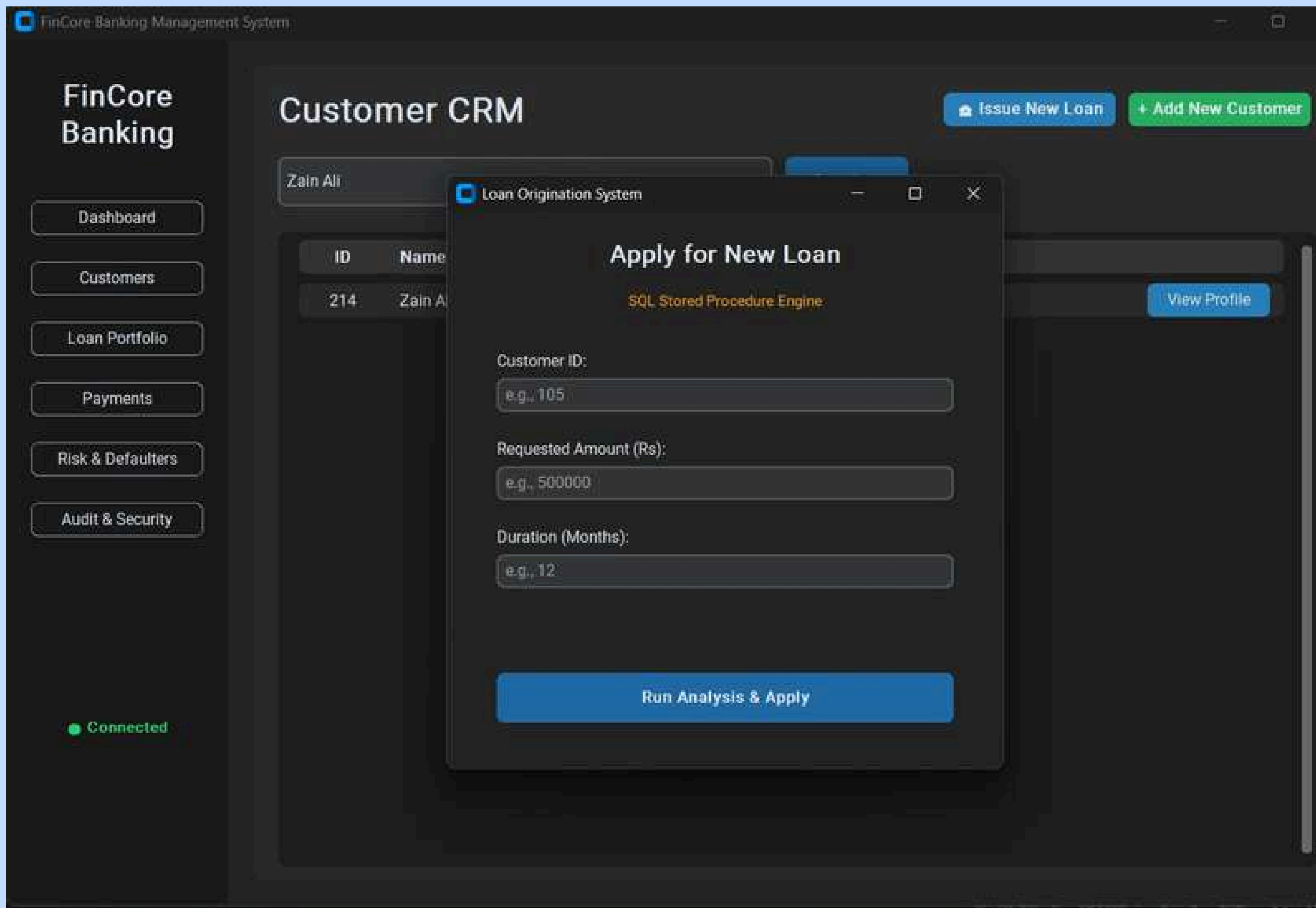
Phone

0310-7509812

V



ISSUING A NEW LOAN



The screenshot displays the 'FinCore Banking Management System' interface. On the left is a sidebar with navigation links: Dashboard, Customers, Loan Portfolio, Payments, Risk & Defaulters, and Audit & Security. The main area is titled 'Customer CRM' and shows a search bar with 'Zain Ali' and a table with columns 'ID' and 'Name'. A modal window titled 'Loan Origination System' is open, featuring the heading 'Apply for New Loan' and the text 'SQL Stored Procedure Engine:'. The modal contains three input fields: 'Customer ID:' with the value 'e.g., 105', 'Requested Amount (Rs):' with the value 'e.g., 500000', and 'Duration (Months):' with the value 'e.g., 12'. A blue button at the bottom of the modal is labeled 'Run Analysis & Apply'. In the top right of the CRM section, there are buttons for 'Issue New Loan' and 'Add New Customer'. A 'View Profile' button is also visible on the right side of the CRM section.

FinCore Banking Management System

FinCore Banking

Dashboard

Customers

Loan Portfolio

Payments

Risk & Defaulters

Audit & Security

Connected

Customer CRM

Issue New Loan

Add New Customer

Zain Ali

ID

Name

214

Zain A

Loan Origination System

Apply for New Loan

SQL Stored Procedure Engine:

Customer ID:

e.g., 105

Requested Amount (Rs):

e.g., 500000

Duration (Months):

e.g., 12

Run Analysis & Apply

View Profile

LOAN PORTFOLIO SECTION

FinCore Banking Management System

FinCore Banking

Dashboard

Customers

Loan Portfolio

Payments

Risk & Defaulters

Audit & Security

Connected

FinCore | Loan Risk & Search

Search Customer Name for Loan Status...

Search Loans

High Risk Portfolio (Top 15 Defaulted Accounts)

ID	Customer	Amount	Status	Action
#420	Bilal Zafar	Rs. 4,560,425	DEFAULTED	History
#1019	Fatima Javed	Rs. 4,555,847	DEFAULTED	History
#799	Zara Zafar	Rs. 4,516,769	DEFAULTED	History
#823	Zainab Butt	Rs. 4,298,356	DEFAULTED	History
#65	Omar Jalil	Rs. 4,062,781	DEFAULTED	History
#540	Bilal Rehman	Rs. 3,970,646	DEFAULTED	History
#882	Nadia Siddiqui	Rs. 3,961,157	DEFAULTED	History
#1002	Iqra Shah	Rs. 3,936,053	DEFAULTED	History
#643	Hamza Mahmood	Rs. 3,932,745	DEFAULTED	History
#584	Rabia Qureshi	Rs. 3,874,490	DEFAULTED	History
#865	Amina Javed	Rs. 3,786,811	DEFAULTED	History
#996	Omar Farooq	Rs. 3,611,431	DEFAULTED	History
#646	Usman Tariq	Rs. 3,545,594	DEFAULTED	History



HISTORY OF CUSTOMER LOANS

[illegible]



PAYMENTS REVIEW

FinCore Banking Management System

FinCore Banking

Dashboard

Customers

Loan Portfolio

Payments

Risk & Defaulters

Audit & Security

● Connected

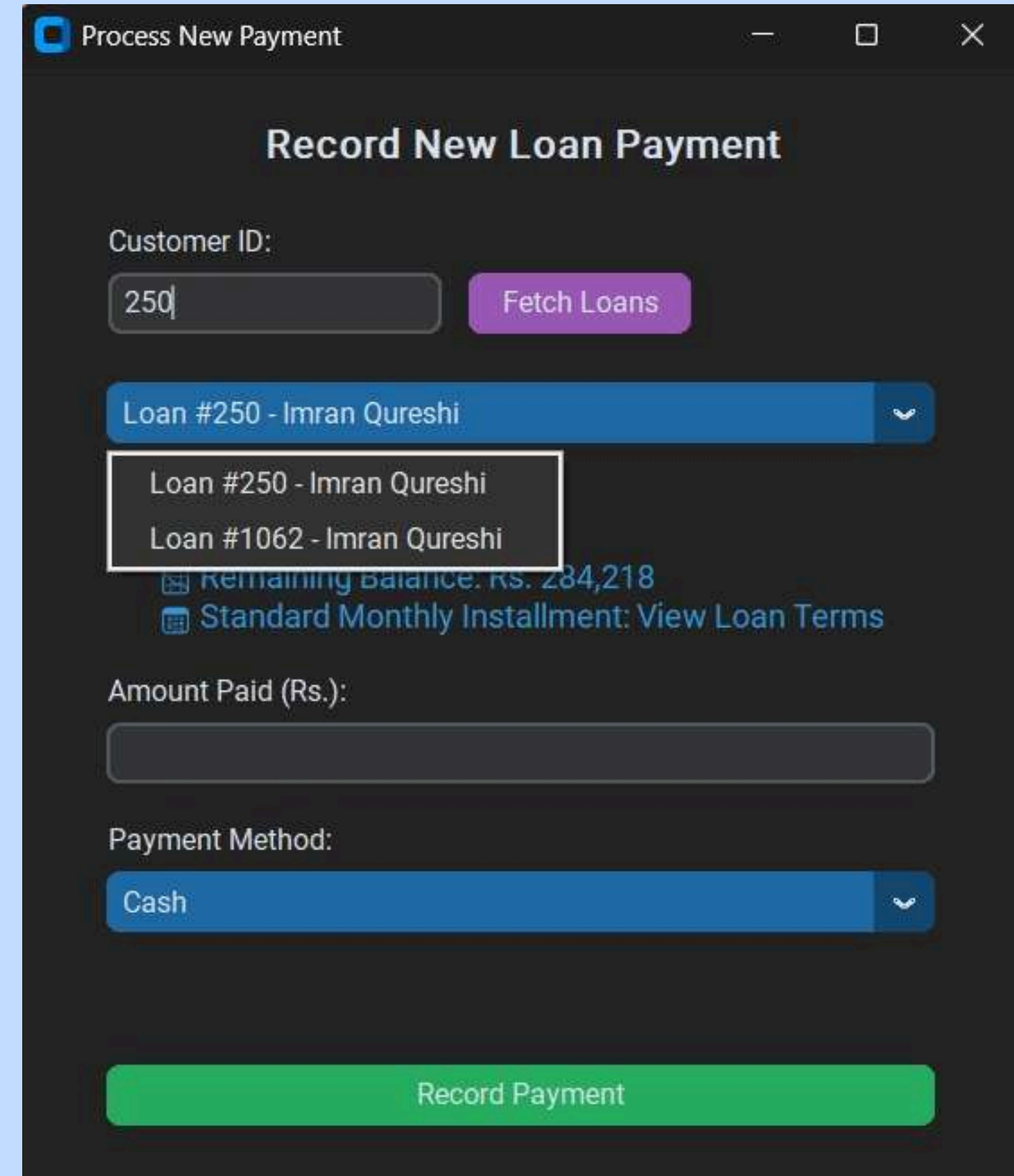
Payment Processing

Process Payment (SP)

Pay ID	Customer	Amount	Date	Method	Fine
#6699	Nasir Khan Jan (L-1065)	Rs. 10,000	2026-05-04	Cash	Rs. 0
#6663	Usman Rehman (L-101)	Rs. 60,000	2026-05-03	Cash	Rs. 0
#3757	Aisha Sheikh (L-602)	Rs. 12,043	2024-01-03	Cheque	Rs. 5,000
#1732	Tariq Jalil (L-271)	Rs. 8,855	2024-01-02	Cash	Rs. 0
#436	Omar Farooq (L-69)	Rs. 51,382	2023-12-31	Cash	Rs. 5,000
#4131	Kashif Javed (L-659)	Rs. 81,520	2023-12-17	Cheque	Rs. 5,000
#1731	Tariq Jalil (L-271)	Rs. 8,855	2023-12-08	Cash	Rs. 5,000
#2022	Nida Baig (L-320)	Rs. 7,090	2023-12-04	Online Transfer	Rs. 5,000
#3756	Aisha Sheikh (L-602)	Rs. 12,043	2023-11-28	Cash	Rs. 0
#435	Omar Farooq (L-69)	Rs. 51,382	2023-11-23	Cheque	Rs. 0
#4130	Kashif Javed (L-659)	Rs. 81,520	2023-11-19	Online Transfer	Rs. 5,000
#1579	Hira Mirza (L-248)	Rs. 54,487	2023-11-18	Cash	Rs. 0
#6549	Meerub Malik (L-1035)	Rs. 26,659	2023-11-16	Online Transfer	Rs. 5,000
#2265	Ali Syed (L-359)	Rs. 33,947	2023-11-09	Online Transfer	Rs. 5,000
#1062	Hamza Ahmad (L-167)	Rs. 183,121	2023-11-09	Online Transfer	Rs. 5,000
#3709	Hassan Farooq (L-594)	Rs. 26,815	2023-11-08	Online Transfer	Rs. 5,000
#2069	Nadia Gillani (L-326)	Rs. 4,032	2023-11-08	Cash	Rs. 5,000

RECORDING PAYMENTS

THE SYSTEM SUPPORTS CONCURRENT LOAN MANAGEMENT, ALLOWING CLIENTS TO MAINTAIN MULTIPLE ACTIVE ACCOUNTS. THE INTERFACE PROVIDES A SELECTION MECHANISM TO ENSURE PAYMENTS ARE ACCURATELY ALLOCATED TO THE USER'S CHOSEN LOAN ID



The screenshot displays a web application window titled "Process New Payment". Inside, there is a section titled "Record New Loan Payment". The form includes the following elements:

- Customer ID:** A text input field containing "250" and a "Fetch Loans" button.
- Loan Selection:** A dropdown menu currently showing "Loan #250 - Imran Qureshi". A tooltip is visible, showing two options: "Loan #250 - Imran Qureshi" and "Loan #1062 - Imran Qureshi".
- Loan Details:** Below the dropdown, there are two lines of text: "Remaining Balance: Rs. 284,218" and "Standard Monthly Installment: View Loan Terms".
- Amount Paid (Rs.):** A text input field.
- Payment Method:** A dropdown menu currently showing "Cash".
- Action:** A large green button labeled "Record Payment" at the bottom.

RISK AND DEFAULTER SECTION

FinCore Banking Management System

FinCore Banking

Dashboard

Customers

Loan Portfolio

Payments

Risk & Defaulters

Audit & Security

Connected

FinCore | Customer Analytics

Prime Customers

Defaulter Watchlist

Risk Classification

Meerub Malik

STATUS: PRIME / PRE-APPROVED

Income: Rs. 154,136.00

Score: 850

Rabia Hashmi

STATUS: PRIME / PRE-APPROVED

Income: Rs. 192,071.00

Score: 850

Hassan Zafar

STATUS: PRIME / PRE-APPROVED

Income: Rs. 238,518.00

Score: 849

Fahad Raza

STATUS: PRIME / PRE-APPROVED

Income: Rs. 366,700.00

Score: 849

Hamza Baig

STATUS: PRIME / PRE-APPROVED

Income: Rs. 122,677.00

Score: 849

Zainab Chaudhry

Income: Rs. 444,769.00

AUDIT AND SECURITY

FinCore Banking Management System

FinCore Banking

Dashboard

Customers

Loan Portfolio

Payments

Risk & Defaulters

Audit & Security

Connected

Security & Audit Logs

Powered by SQL Triggers

Log ID	Date/Time	Action Performed	Table (Rec ID)	Old Value	New Value
#5	2026-05-04 00:25	INSERT - New Loan Issued	LOAN (1065)	NULL	Amount: 120000...
#2	2026-05-02 14:29	UPDATE - Reschedule Status	LOAN (5)	0	1
#1	2026-05-02 14:29	UPDATE - Approved Amount	LOAN (5)	3873270.50	50000.00

End of Log



Thank you for your attention
Any Questions?

